dbt without the warehouse (or the bill)

DuckDB, end to end — a dbt project developed, tested, deployed, and served entirely on DuckDB.

[DuckDB duck mark goes here] · dbt + DuckDB

We rented a distributed system to transform a few gigabytes.

Every dbt run is a round-trip you didn't need

- Slow
- Metered — you pay by the second
- Queued behind your colleagues' CI jobs

If your whole project fits in a few hundred GB...

...this is a strange way to live. You can stop.

DuckDB

- Free, MIT-licensed, in-process analytical database
- No server. No account. No invoice.
- Now in public beta on the dbt Fusion engine

What that unlocks

- Sub-second model iteration on your laptop
- Full-project CI in seconds on a plain GitHub runner
- Python models that run in-process — your data never moves

The warehouse becomes optional — at every stage

- Native readers: Parquet, Iceberg, Postgres — read sources in place
- DuckLake: publish your output tables in open formats
- DuckDB-WASM: query them straight from the browser
- Optional at every stage — not just development

Let's build one. Live.

Developed → Tested → Deployed → Served, entirely on DuckDB

~3M NYC taxi trips — more data than you'd think reasonable, on one laptop

1 · Develop

```
$ dbt run
```

- stg_trips — reads Parquet straight from a URL, no load step
- payment_summary — a normal mart via ref()
- trips_by_hour.py — a Python model, run in-process

2 · Test

```
$ dbt test
```

- Schema tests: not_null, unique, accepted_values
- One business assertion, over ~3M rows:

  Credit-card riders tip $4.17 (28.7%). Cash riders: $0.00.

3 · Deploy

```
$ # the publish models wrote open Parquet during `dbt run`
```

- publish_* models are materialized as external Parquet
- Your output tables, in an open format — no warehouse to load
- DuckLake does exactly this at production scale

4 · Serve

```
$ cd web && python3 -m http.server 8000
```

- Open http://localhost:8000
- DuckDB-WASM queries the published Parquet in the browser
- No backend. No server. The page IS the database.

The receipts

- ~3,000,000 rows
- Developed + tested in seconds — on this laptop
- The DuckDB scan itself: milliseconds
- Total cloud bill: $0.00

Your data is not that big.

Your bill doesn't have to be either.

github.com/…/duckdb-samples · dbt-scenarios/end-to-end-duckdb